

数字系统课程设计实验报告

基于Verilog的密码锁控制器

1 功能需求描述

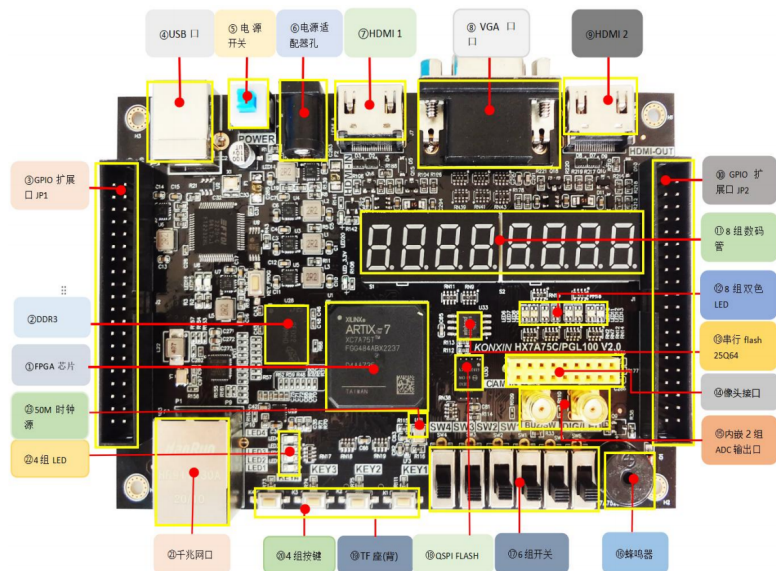
使用FPGA模仿密码锁的工作过程，完成密码锁的核心功能，需求如下：

- 管理员可以通过设置（专用按键）更改密码
- 如果没有预置密码，密码缺省为“0000”
- 用户如果需要开锁，拨动相应的开关进入输入密码状态，输入4位密码，按下确定键后，若密码正确，锁打开，若密码错误，将提示密码错误，要求重新输入，三次输入都错误，将发出报警信号
- 报警后，只有管理员作相应的处理（专用按键）才能停止报警
- 用户输入密码时，在按下确定键之前，可以通过按退格键修正，每按一次退格键消除一位密码
- 正确开锁后，用户处理完毕后，按下确定键，系统回到等待状态
- 系统操作过程中，只要密码锁没有打开，如果10秒没有对系统操作，系统回到等待状态
- 系统操作过程中，如果密码锁已经打开，如果20秒没有对系统操作，系统自动上锁，回到等待状态

3 需求分析

3.1 硬件与软件环境

本实验使用HX7A75C开发板，在Vivado 2023.2环境下进行开发。



本实验使用了开发板上的全部四个按键和四个开关用于输入，具体安排如下：

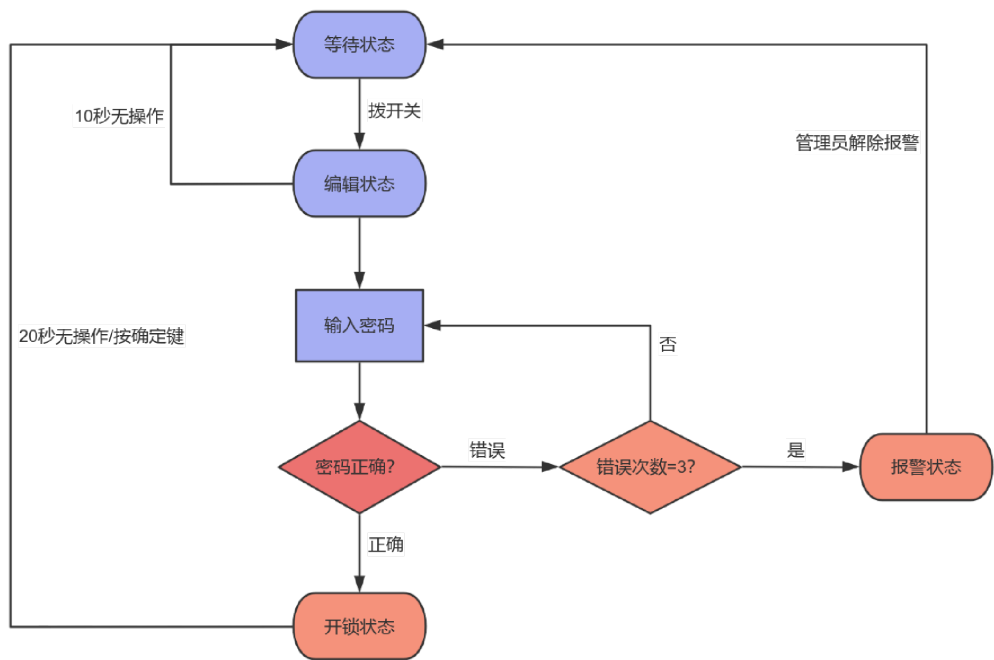
变量名	对应按键/开关	功能概述
switches	SW4~1	用BCD码表示一位十进制数
key_button	KEY4	报警时，管理员按下此按键解除报警
ok_button	KEY3	输入完毕后按下以确认
admin_button	KEY2	在用户、管理员之间切换
load	KEY1	进位、退位键

输出的硬件安排见[4.2 模块化设计](#)中的状态显示和数码管显示小节。

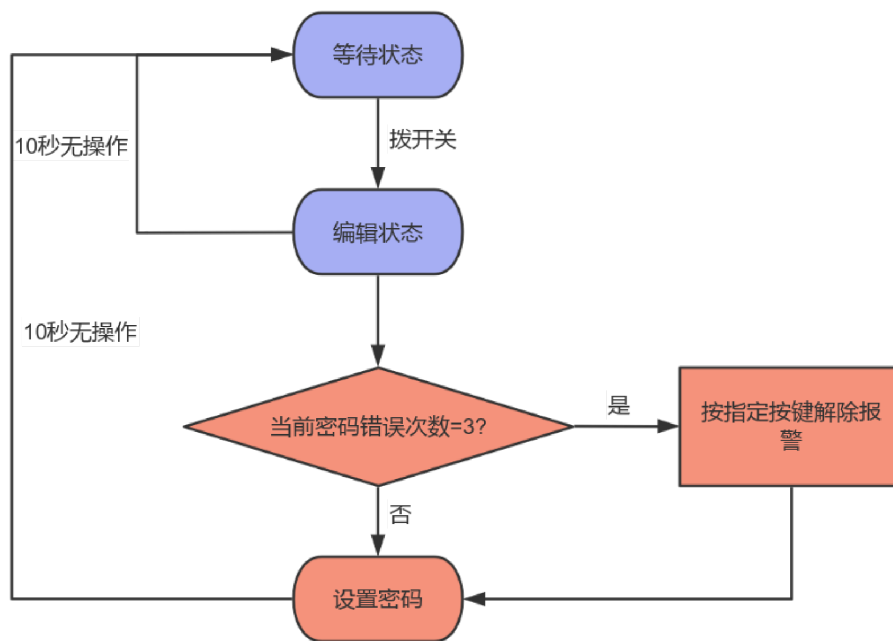
4 方案

4.1 流程图分析

- 用户模式下的流程图：



- 管理员模式下的流程图：



我们将两种模式下的状态切换逻辑統合到接下来顶层模块状态机的设计中。

4.2 模块化设计

我们将密码锁的设计分为如下几个模块：

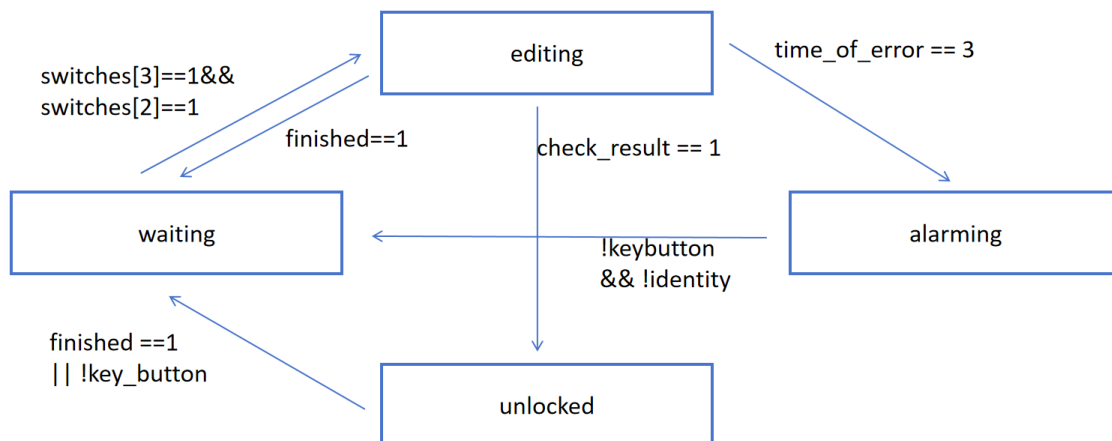
1. **状态机**。密码锁可以划分为四个状态：等待状态(waiting)、编辑状态(editing)、报警状态(alarming)、开锁状态(unlocked)；以及两种模式：用户模式(user)、管理员模式(admin)。其中编辑状态是指用户模式下可以输入密码的状态以及管理员模式下可以修改密码的状态。在代码中，通过如下语句对这些状态和模式进行了编码：

```

parameter waiting = 2'b00;
parameter editing = 2'b01;
parameter unlocked = 2'b10;
parameter alarming = 2'b11;
parameter user = 1'b1;
parameter admin = 1'b0;

```

在主体模块中，需要实现一个状态机来控制状态的切换。状态转换图*如下：



具体说明如下：

- waiting→editing: 拨动特定的两个开关进入编辑状态。
- editing→waiting: 10秒无动作回到等待状态，这里 `finished` 信号表示计时时间到。
- editing→unlocked: 输入密码正确，开锁。
- editing→alarming: 输入错误三次报警。
- alarming→waiting: 管理员模式下按下指定按键解除报警。
- unlocked→waiting: 开锁后二十秒无操作自动关锁。

2. **密码的输入与存储。**在密码锁中，需要存储两组四位密码，一组是用户输入的密码，另一组是管理员置入的正确密码。我们使用两个**十六位移位寄存器**来进行密码的存储。密码的输入使用四个开关来表示一位BCD码，开关倒向上代表0，开关在下代表1，如0011表示3，另外有一个 `load` 按键，按下此按键时就将当前四个开关代表的四位二进制数加载进移位寄存器里(即左移四位)。对于**退位功能**，我们原先想另外分配一个按键专门用于退位，但是所给FPGA板的按键十分有限，我们只能复用 `load` 键来进行退格功能。我们规定，如果当前BCD码为有效值(0~9)，则按 `load` 键加载；如果当前BCD码为无效值(A~F)，则按 `load` 键退格，下面是一个例子：

假设当前寄存器中存储16'd1145:

- 如果开关为0001，表示下一位是1，按 `load` 键，则寄存器变为16'd1451；
- 如果开关为1111，是无效的BCD码，按 `load` 键，则寄存器变为16'd0114。

3. **密码检查与错误次数记录。**需要实现一个密码检查模块，当用户输入完密码后，按下确定键 `ok`，就将用户密码寄存器的数据和管理员正确密码寄存器中的数据进行比较，如果相等，则开锁，如果不相等，则记录一次错误并停留在编辑状态。当错误次数达到三次时，触发报警。

4. **计时器与计时中断。**需要实现一个计时器，也就是由clk方波信号驱动的计数器，它可以当前的状态决定计时长度的，在编辑状态下，计时到10秒时切回等待状态，在开锁状态下，计时到20秒时切回等待状态。此外，在计时时如果有任何操作（如拨开关、按按钮等），都要把计时清零。

5. **状态显示。**我们将当前的状态通过LED灯表示。所给的FPGA板有四个LED灯，我们规定：

- 一个灯亮：等待状态
- 两个灯亮：编辑状态
- 三个灯亮：报警状态
- 四个灯亮：开锁状态

6. 数码管显示。所给的FPGA板上有八个七位数码管，我们规定*：

- 前两位：显示当前计时秒数
- 第三位：显示当前错误次数
- 第四位：显示当前身份，0为管理员，1为用户
- 后四位：显示当前密码寄存器的内容，根据当前模式决定显示用户密码或是正确密码

在具体实现中，为方便起见，将这一需求与密码寄存器在同一个模块中实现。

4.3 顶层及子模块功能、接口描述

- 顶层模块 `system_logic`：负责控制密码锁的状态切换，包含了各个子模块的实例化，是系统的顶层模块。该模块的接口描述如下：

```
module system_logic ( // manages the overall logic of the lock, **it should be the top
module**
    input clk, // the clock signal
    input [3:0] switches, // represent a decimal 0~9
    input load, // load one digit or delete one digit
    input ok_button, // confirm button
    input admin_button, // to switch between user/admin mode
    input key_button, // when unlocked, push this to return waiting
    output [6:0] tubes, // LED tubes that display the passwords you entered
    output [3:0] LEDs, // the LEDs that indicates the state of the lock
    output [7:0] sel // select the digit you want to display
);
```

该模块中的状态机代码如下，这些判断语句实现了[状态转换图](#)中所有状态转换：

```
always @(posedge clk)
begin
    next_state = state; // to avoid generating latch
    if (state == waiting && (switches[3] && switches[2]))
        begin // 等待状态下，拨动两个开关进入编辑状态
            next_state = editing;
        end
    if (state == editing && finished == 1)
        begin // 10s no operation, back to waiting
            next_state = waiting;
        end
    if (state == editing && check_result == 1 && !ok_signal)
        begin // when you entered the right password
            next_state = unlocked;
        end
end
```

```

if (state == unlocked && finished == 1)
    begin // 20s no operation,back to waiting
        next_state = waiting;
    end
if (state == editing && time_of_error == 2'd3)
    begin // when you entered wrong password 3 times
        next_state = alarming;
    end
if (state == alarming && !ok_signal &&!identity )
    begin // 管理员模式下按确定键解除报警
        next_state = waiting;
    end
if (state == unlocked && !key_button )
    begin //
        next_state = waiting;
    end
end
always @(posedge clk)
    begin
        state <= next_state;
    end
assign state_out = state;
endmodule

```

- **密码寄存器模块** `password_reg`: 负责存储、输入、修改和显示密码，同时还读取当前模式(user/admin)、当前错误次数、当前计时秒数并一起显示。

```

module password_reg(
    input clk,
    input identity,    // user'1' / admininstrator'0'
    input load, // to load one digit (left-shift), posedge trigger
    input[3:0] one_digit, // the one decimal digit you input
    input[1:0] time_of_error,
    input[31:0] cnt_1s,
    input ok,
    output reg [15:0] q, // the shift reg that loads password
    output reg [15:0] new_pswd, // the new password you set,when"identity==0"
    output reg [6:0] tubesreg, // the tubes to display the first decimal digit
    output reg [7:0] sel //
);

```

为了减少 FPGA 芯片 IO 口的使用数量，一般会采用分时复用的扫描显示方案进行数码管驱动。我们使用 `sel` 来分时显示8个七段数码管的内容。控制信号的刷新频率必须足够快才能避免闪烁感，但也不能太快，以免影响数码管的开关切换，最佳工作频率为 1000Hz 左右。我们使用如下循环，每1ms刷新一次 `sel` 的内容：

```

always @(posedge clk)
begin
    if(cnt_1ms == 49_999)                //计时1ms
        cnt_1ms <= 0;                    //清零计时
    else                                  //计时未到
        cnt_1ms <= cnt_1ms + 1;          //继续计时
    if (!load)
        sel <= 8'b11111110;
    else if(cnt_1ms == 49_999)            //计时到
        sel <= {sel[6:0],sel[7]}; //循环左移
    else
        sel <= sel;
end

```

本项目使用的开发板中，数码管是共阳极的，所以要让某一段亮起，则该位应为低电平。在最后输出时，根据 case 语句判断当前状态，将对应段码输出到数码管上。

- 密码检查模块 `password_check`: 当用户按下确定键时，更新一次比较结果和错误次数。

```

module password_check (
    input identity, // user'1' / admin '0' , only when identity==1 will this module be
    enabled
    input ok_signal, // when posedge, check the password
    input [15:0] password, // the password you input
    input [15:0] correct_pswd, // the correct password
    output reg result, // '1': right '0': wrongs
    output reg [1:0]time_of_error // records the time you enter wrong password
);

```

- 计时模块 `timer`:

```

module timer (
    input[1:0] state, // 4 states: 00 waiting, 01:editing, 10:unlocked, 11:alarming
    input ok, // press the button to transfer to waiting mode when unlocked
    input clk, // a standard square wave signal
    input [3:0]switches,
    input load,
    input key,
    input admin_button,
    output reg [31:0]cnttemp,
    output reg finished
);

```

我们定义了两个变量：cnt_1ms和cnt_1s进行级联计时，每过50000个时钟周期，cnt_1ms自增1，每cnt_1ms每增加1000，cnt_1s增加1。这样cnt_1s就实现了1秒的计时：

```

always @(posedge clk)
begin
    if(!key||!load||!admin_button||!ok||lastone!=switches) // 进行任何操作都会打断计时
    begin
        cnt_1s<=0;
        cnt_1ms<=0;
    end
    cnttemp = cnt_1s;

```

```

if(cnt == 32'd49999)
begin
    cnt_1ms <= cnt_1ms+1;
    cnt<=0;
end
else
    cnt <= cnt + 1;
if(cnt_1ms == 32'd1000)
begin
    cnt_1s <= cnt_1s+1;
    cnt_1ms<=0;
end
if (cnt_1s==32'd10&&state==editing)
begin
    finished <= 1'b1;
    cnt_1s <= 32'd0;
end
else if (cnt_1s==32'd20&&state==unlocked)
begin
    finished <= 1'b1;
    cnt_1s<=32'd0;
end
if(state==waiting)
    finished <= 1'b0;
end
endmodule

```

- 状态指示模块 `state_indicate`:

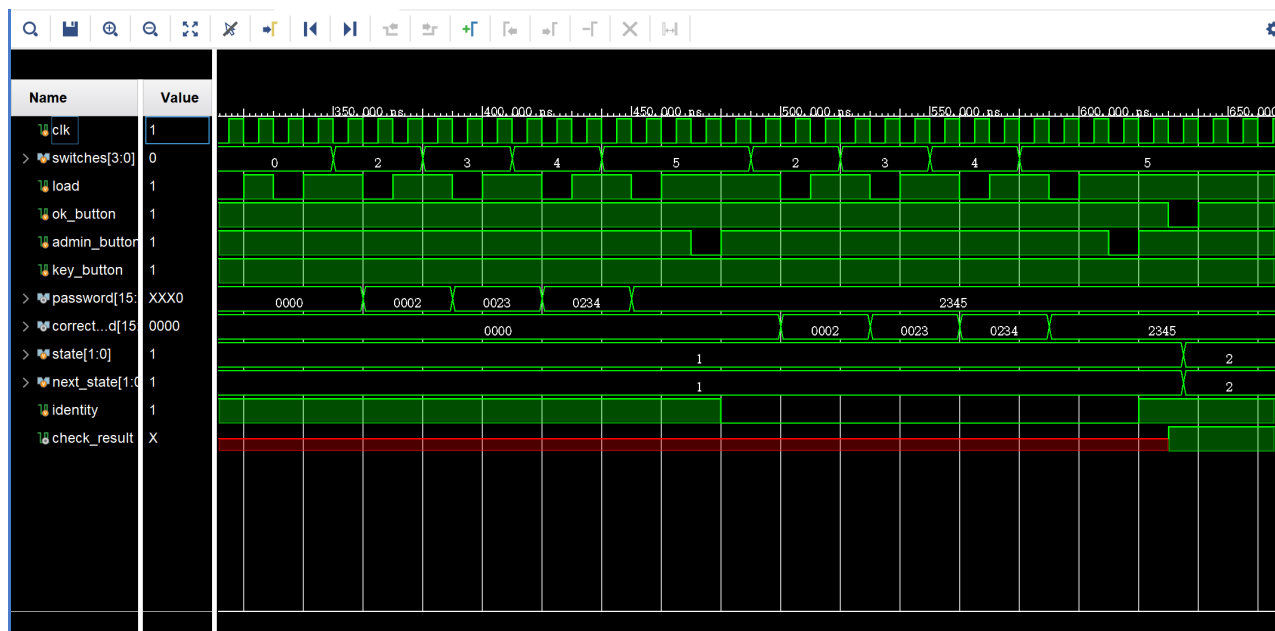
负责根据当前状态控制4个LED灯的显示，详见前文对[数码管显示](#)的规定。

```

module state_indicate ( // control the LEDs
    //ports
    input [1:0] state,
    output reg [3:0] LEDs
);

```

5 仿真测试



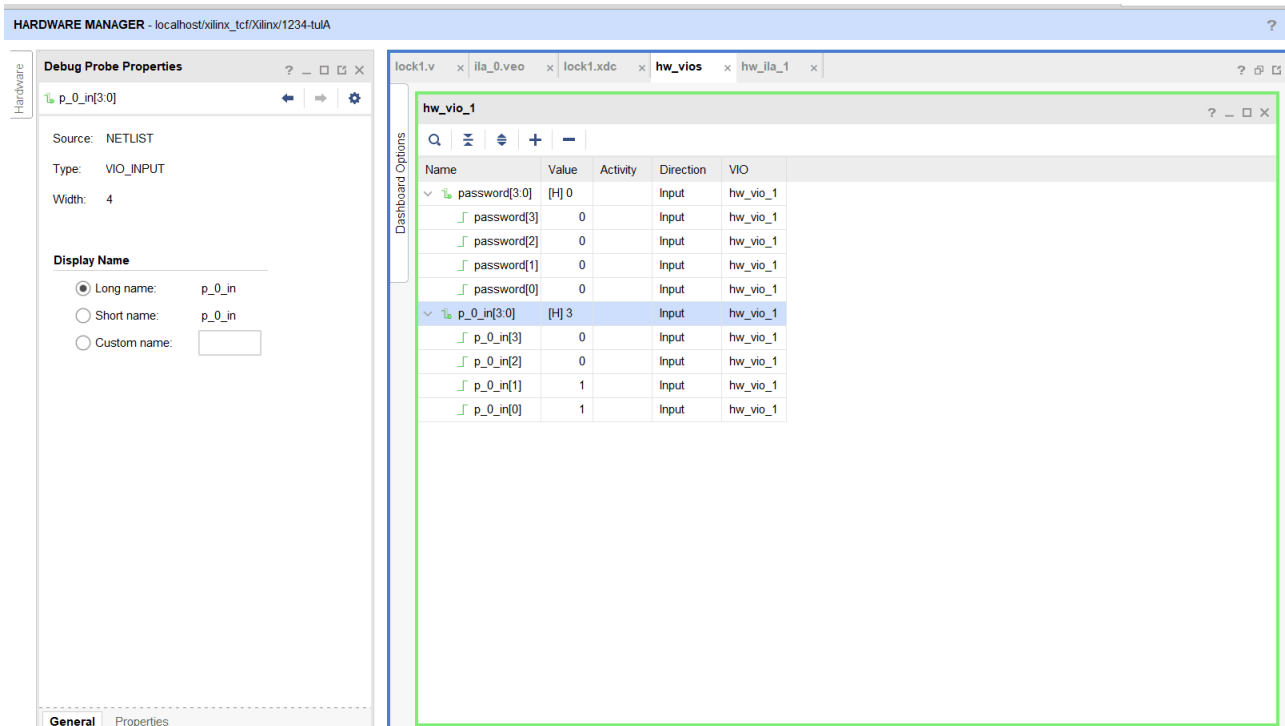
- 如图所示，我们先进入用户模式输入密码2345，再进入管理员模式将正确密码设置为2345，从 `password_reg` 和 `correct_password` 的变化可以看出密码寄存器正常工作。
- 最后按下 `ok_button` 进行确认，此时 `check_result` 变为1，表明密码比较模块正常工作，同时状态 `state` 由1(editing)变为2(unlocked)，表明状态机模块正常工作。

通过仿真，我们初步验证了密码锁可正常工作。

6 VIO/ILA调试

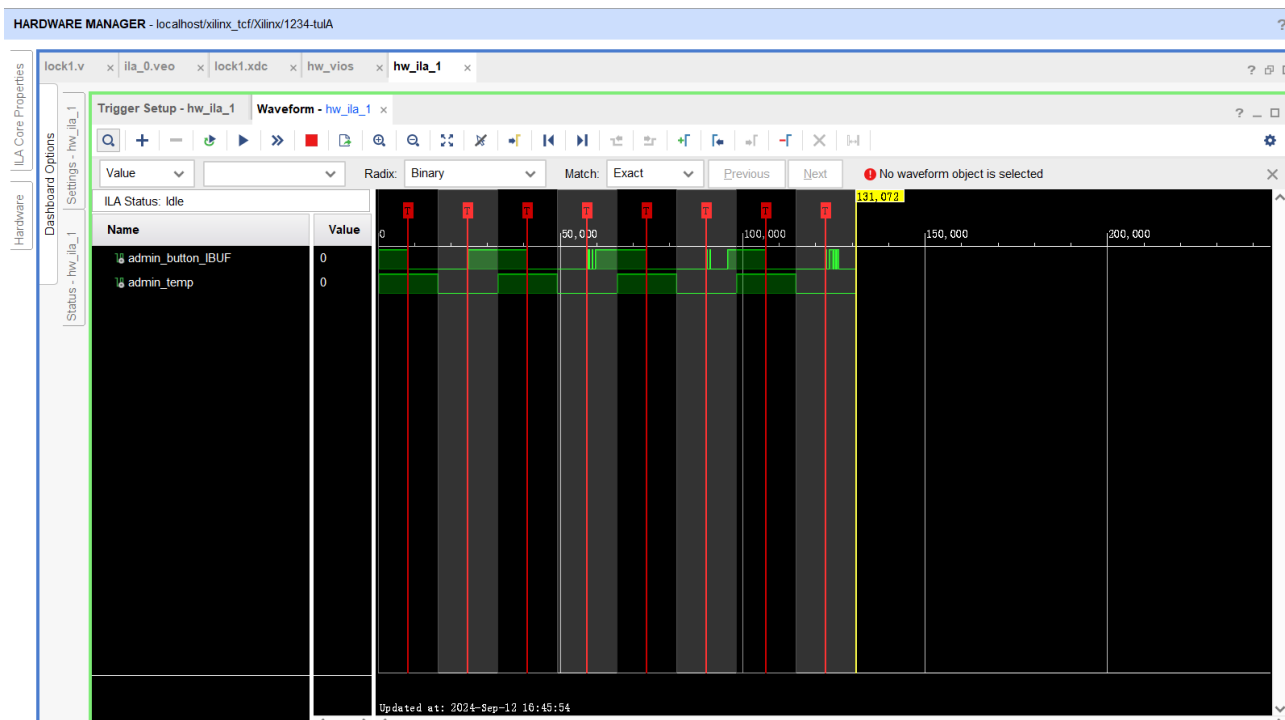
6.1 VIO核

配置了vio核，查看变量password以及密码寄存器内部的q变量，确保 `password_reg` 模块是能够正常输入密码并且存储的。



6.2 ILA核

为了验证消抖模块的可靠性，同时调整消抖模块内部的参数，决定消抖的时长，配置了ila核，采样的数据深度为128K，即131072。采样位宽均为1，设置为下降沿触发。对消抖之前的按键信号和消抖之后的按键信号进行抓取，得到的信号波形如下图所示。按键消抖模块正常工作，确保整个系统的正常工作。



7 遇到的问题和解决方法

7.1 多重驱动问题

在刚开始写代码时常常遇到“multi-driven”的 error，经查阅资料，发现是在多个 always 块里对同一个变量赋值导致的。最终我们总结出两个解决方法：

- 尽量在一个 always 块里完成赋值；
- 可以借用标志信号，在满足条件时将其置 1，然后另一个 always 块里判断标志信号是否为 1，若为 1 则执行。这两个方法帮助我们解决了大多数的多重驱动问题。

7.2 阻塞赋值与非阻塞赋值问题

如果 always 块是由边沿触发的，通常需要采用非阻塞赋值的方法，如果是电平触发的，需要阻塞赋值，而且同一个变量不能两种赋值都使用，会有警告，避免出现时序问题。

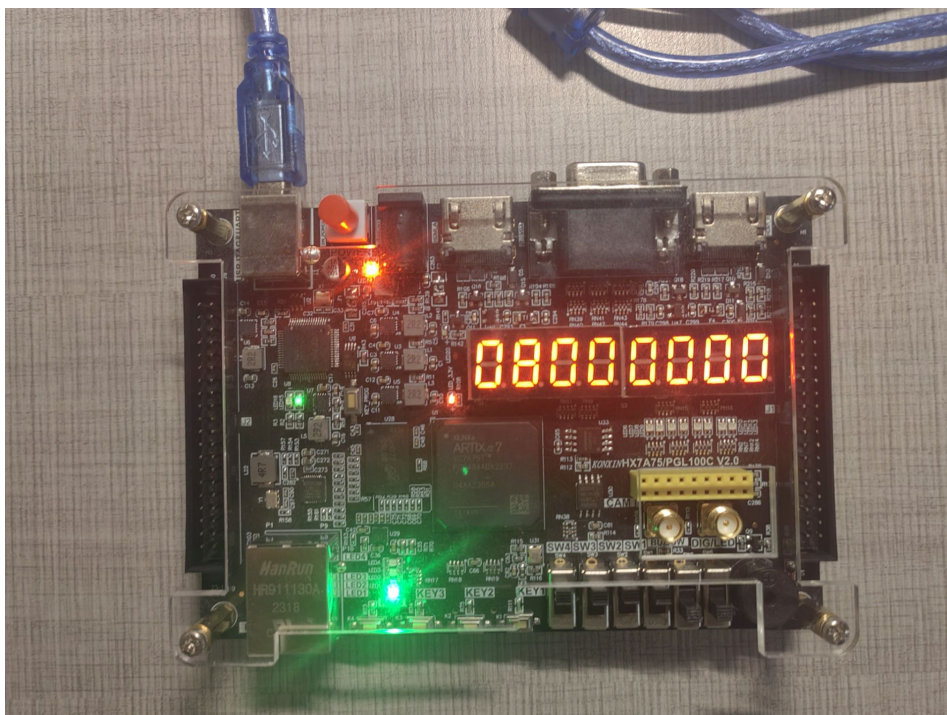
7.3 电路锁存器警告

在使用组合逻辑赋值的时候，如果没有考虑到所有的情况，电路会生成一个 latch，可能会导致电路出现问题，在全部语句之后加一个 else 如果情况都不满足，将自己的值赋给自己就可以解决这个警告。

8 实物展示

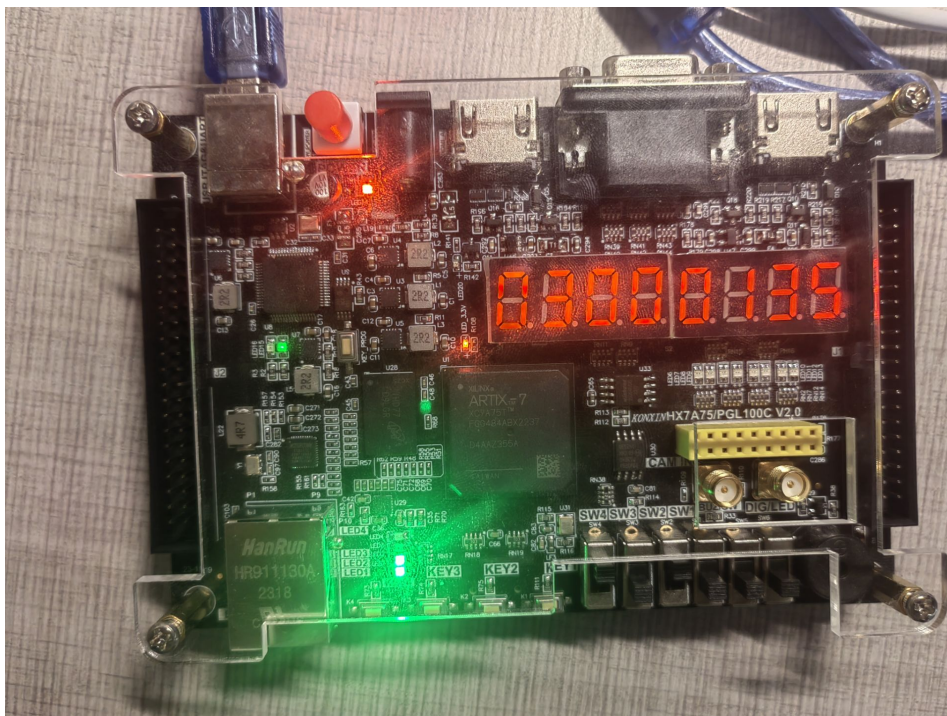
8.1 自动计时

如图，上电后，拨动开关进入编辑状态，此时自动计时10秒，若10秒没有如何操作，将回到等待状态。



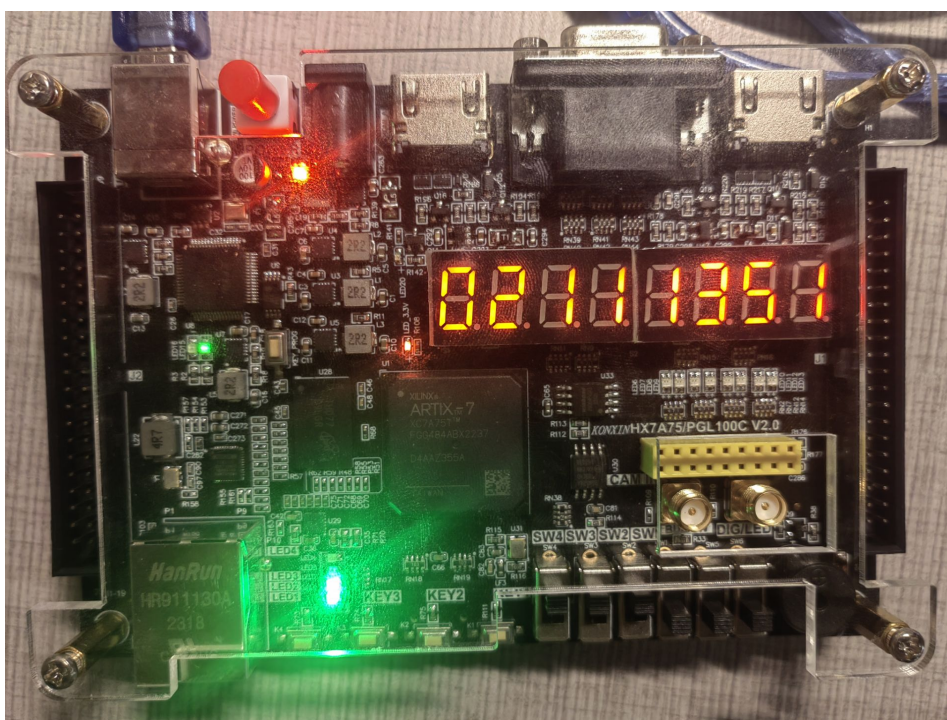
8.2 设置密码

编辑状态下，管理员设置密码0135。

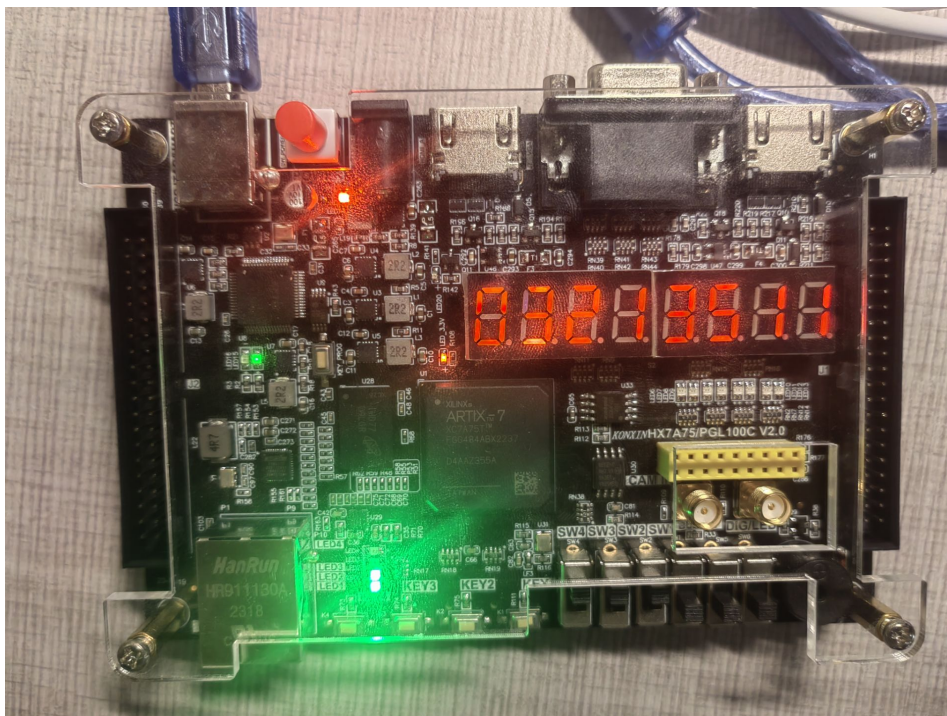


8.3 错误次数统计与报警

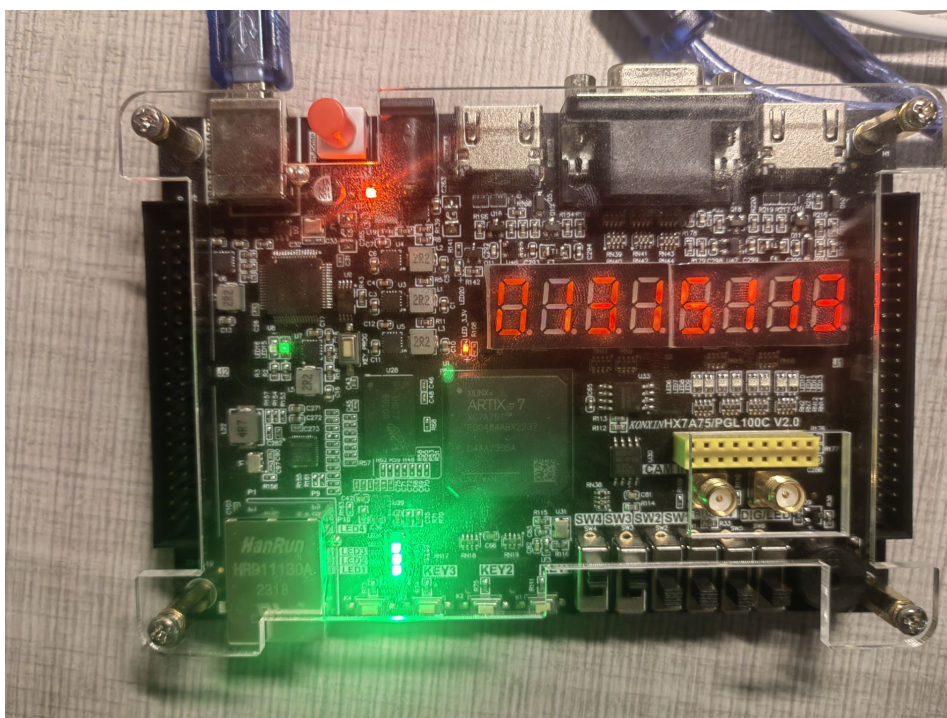
- 用户输入错误密码，错误次数变为1（数码管的第三位显示错误次数）



- 用户输入错误密码，错误次数为2

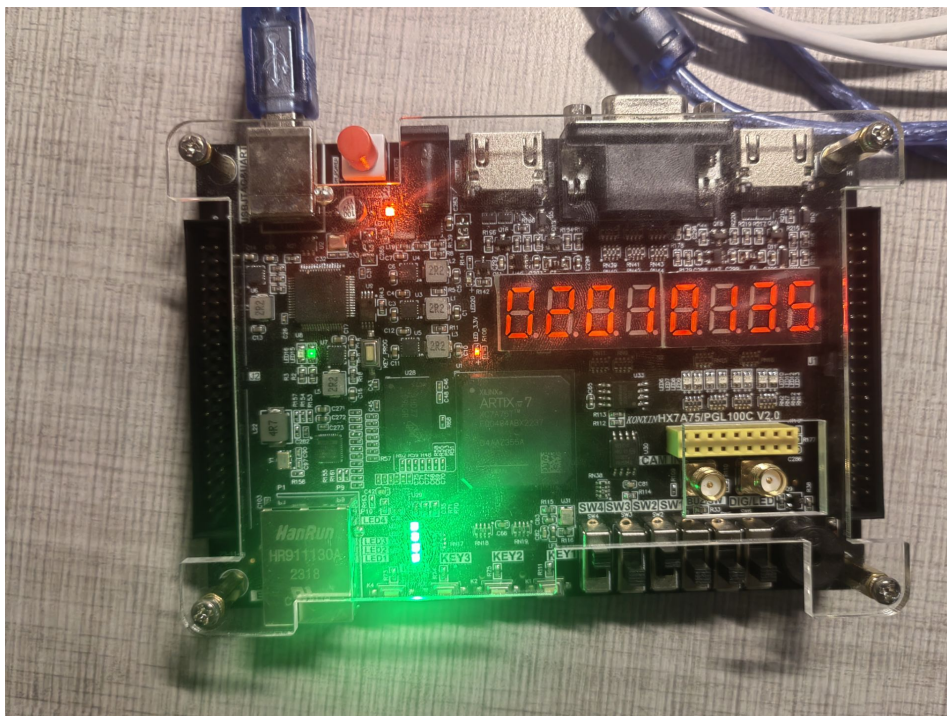


- 用户再次输入错误密码，错误次数为3，并报警（三盏灯亮）



8.4 开锁

用户输入正确密码，开灯，四个灯全亮。



9 心得与体会

在这次课程设计中，我组两名成员均为第一次接触 Verilog。从一开始接触并行语言，再到第一次下载程序，然后第一次仿真看到时钟波形，第一次使用vio核和ila核，第一次固化程序到板子内部，这无数件小事，都让我们产生了满满的成就感。

在整个开发过程中，我们遇到了很多困难，例如多重驱动报错、wire 和 reg 混用报错、阻塞赋值与非阻塞赋值的问题，但最终在csdn和gpt的帮助下，问题也都得以解决。

在这一次课程中，我们认真学习了 Verilog 语法，利用课余时间 HDLbits 上刷了一定量题目，清楚了基本的法知识，基本清楚了如何用 Verilog 语言来描述具有一定组合逻辑和时序逻辑的电路，增强了对数字电路知识的理解，也学习了 FPGA 的使用方法。

虽然这次项目的程序量不大，包括测试文件在内的代码量才接近1000行，其中还有一些代码的编写有重复之嫌，但是在本次项目中我们将代码上传到GitHub上进行了版本的管理，使得团队协作之间有了更高的效率。因此整个项目程序从开始到上板验证成功，总共也就一天左右的时间。编写程序的过程锻炼我们的编程逻辑以及上网查阅相关资料的能力，调试程序的过程也锻炼了我们的耐心。相信这次经历一定能为以后的开发打下良好的基础。